LINEAR ALGEBRA
FOR DEEP LEARNING

A Complete Beginner's Course

Using NumPy & Matplotlib — From First Principles

6 Modules	50+ Concepts	100+ Code Examples	NumPy + Matplotlib

MODULE 0

Setup & Orientation

Getting your tools ready before we do any math

Before learning any mathematics, we need to set up our environment. This course uses two libraries: NumPy for all numerical computation, and Matplotlib for visualising what the math means visually. Both are essential for deep learning.

Installing the Libraries

```
# In your terminal or Jupyter notebook:
pip install numpy matplotlib

# Verify installation
python -c "import numpy as np; import matplotlib; print('All good!')"
```

The Standard Imports — Memorise These

Every single file you write in deep learning or data science will start with these exact lines. They are convention, not optional:

```
import numpy as np          # 'np' is the universal alias — never change it
import matplotlib.pyplot as plt  # 'plt' is the universal alias — never change it

# When working in Jupyter notebooks, add this line to show plots inline:
%matplotlib inline
```

WHY The aliases np and plt are used by every data scientist on Earth. If you use a different alias, no one reading your code will understand it — and online tutorials won't match your code.

Your First NumPy Check

```
import numpy as np

print(np.__version__)       # Check what version of NumPy you have
print(np.array([1, 2, 3]))  # Create and print your first array

# Output:
```

```
# 1.26.0   (or similar)
# [1 2 3]
```

Your First Matplotlib Plot

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [1, 4, 9, 16, 25]

plt.plot(x, y, color='navy', linewidth=2)  # Draw the line
plt.title('My First Plot')                 # Add a title
plt.xlabel('x')                            # Label the x-axis
plt.ylabel('y = x squared')               # Label the y-axis
plt.grid(True)                             # Show gridlines
plt.show()                                 # Display the plot
```

Key rule for Matplotlib: Always end your plotting code with `plt.show()` (or it may not display outside Jupyter). Always label your axes — unlabelled axes are meaningless in science and engineering.

MODULE 1

Scalars & Vectors

The building blocks of all deep learning mathematics

1.1 What Is a Scalar?

> **Scalar**
> A single number. Nothing more. It has magnitude (size) but no direction. Temperature, age, price — all scalars.

In mathematics we write a scalar as a lowercase letter: x = 5. In NumPy:

```
import numpy as np

x = 5            # A plain Python int — also a scalar
x = 5.0          # A plain Python float — also a scalar
x = np.float64(5.0)  # A NumPy scalar — same value, numpy type

print(type(x))  # <class 'numpy.float64'>
print(x * 2)    # 10.0
print(x ** 2)   # 25.0
```

> **Deep Learning Connection**
> In neural networks, the learning rate is a scalar (e.g. 0.001). Loss values are scalars. Bias terms are scalars. You will work with scalars constantly.

1.2 What Is a Vector?

> **Vector**
> An ordered list of numbers. It has both magnitude (length) and direction. Think of it as an arrow pointing somewhere in space.

A vector with 3 components lives in 3-dimensional space. A vector with 784 components lives in 784-dimensional space (like a flattened 28×28 image in MNIST). We write a vector as a bold lowercase letter: v.

```
import numpy as np
```

```
# A 1D NumPy array IS a vector
v = np.array([3, 4])        # 2D vector: x=3, y=4
u = np.array([1, 2, 3])     # 3D vector
w = np.array([0.1, -0.5, 2.0, 1.3])  # 4D vector

print(v.shape)    # (2,)  — 2 elements, 1 dimension
print(u.shape)    # (3,)  — 3 elements, 1 dimension
print(v.ndim)     # 1
print(v.dtype)    # int64 (or int32 on Windows)
```

Indexing a Vector

Indexing works identically to Python lists — zero-based:

```
v = np.array([10, 20, 30, 40, 50])

print(v[0])     # 10   — first element
print(v[-1])    # 50   — last element
print(v[1:4])   # [20 30 40] — slice (stop is exclusive)
print(v[:3])    # [10 20 30]
print(v[::2])   # [10 30 50] — every second element
```

Row vs Column Vectors

A row vector is shape (n,) or (1, n). A column vector is shape (n, 1). The distinction becomes critical in matrix multiplication:

```
v = np.array([1, 2, 3])       # Shape (3,) — ambiguous: row or column?

row = v.reshape(1, 3)          # Shape (1, 3) — explicitly a row vector
col = v.reshape(3, 1)          # Shape (3, 1) — explicitly a column vector

print(row.shape)   # (1, 3)
print(col.shape)   # (3, 1)

# Shorthand for column vector:
col = v[:, np.newaxis]         # Same as reshape(3,1)
print(col)
# [[1]
#  [2]
#  [3]]
```

1.3 Vector Operations

Addition — Component-wise

To add two vectors, add each corresponding component. Vectors must have the same length.

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

result = a + b
print(result)   # [5 7 9]

# What NumPy does internally:
# [1+4, 2+5, 3+6] = [5, 7, 9]
```

Scalar Multiplication — Scale Every Component

```
v = np.array([2, 4, 6])
scalar = 3

result = scalar * v
print(result)   # [ 6 12 18]

# Geometrically: the vector stretches 3x longer, same direction
# With a negative scalar, it flips direction:
print(-1 * v)   # [-2 -4 -6]
```

Visualising Vector Addition with Matplotlib

```
import numpy as np
import matplotlib.pyplot as plt

a = np.array([3, 1])
b = np.array([1, 3])
c = a + b        # [4, 4]

fig, ax = plt.subplots(figsize=(6, 6))

# quiver(x_start, y_start, x_length, y_length)
ax.quiver(0, 0, a[0], a[1], angles='xy', scale_units='xy', scale=1,
          color='navy', label='a = [3,1]')
ax.quiver(0, 0, b[0], b[1], angles='xy', scale_units='xy', scale=1,
          color='gold', label='b = [1,3]')
ax.quiver(0, 0, c[0], c[1], angles='xy', scale_units='xy', scale=1,
          color='red', label='a+b = [4,4]', linestyle='dashed')

ax.set_xlim(-1, 6)
ax.set_ylim(-1, 6)
```

```
ax.axhline(0, color='gray', lw=0.5)
ax.axvline(0, color='gray', lw=0.5)
ax.grid(True, alpha=0.3)
ax.legend(fontsize=12)
ax.set_title('Vector Addition: a + b = c')
plt.show()
```

1.4 The Dot Product — The Most Important Vector Operation

> **Dot Product**
>
> Takes two vectors of the same length and produces a single scalar. Written as a · b or a^T b. It is the engine behind neural network layers.

Formula from first principles: if a = [a1, a2, a3] and b = [b1, b2, b3], then:

a · b = a1×b1 + a2×b2 + a3×b3

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Method 1: explicit formula
manual = a[0]*b[0] + a[1]*b[1] + a[2]*b[2]
print(manual)          # 32

# Method 2: np.dot (standard)
print(np.dot(a, b))    # 32

# Method 3: @ operator (Python 3.5+, preferred in deep learning)
print(a @ b)           # 32

# They all give the same answer: 1*4 + 2*5 + 3*6 = 4+10+18 = 32
```

What does the dot product measure geometrically?

The dot product measures how much two vectors point in the same direction:

```
a = np.array([1, 0])   # Points right
b = np.array([0, 1])   # Points up (perpendicular to a)
c = np.array([1, 0])   # Points right (same as a)

print(np.dot(a, b))    # 0.0  — perpendicular vectors: dot product = 0
print(np.dot(a, c))    # 1.0  — parallel vectors: dot product = maximum
print(np.dot(a, -c))   # -1.0 — opposite direction: dot product < 0
```

Deep Learning Connection

Every neuron in a neural network computes a dot product: output = weights · inputs + bias. If you have 784 inputs (one per pixel) and 256 neurons, that is 784 × 256 = 200,704 dot products — all done simultaneously with NumPy's vectorised matrix multiplication.

1.5 Vector Norm — The Length of a Vector

Norm

The 'length' of a vector. The most common is the L2 norm (Euclidean length): $||v|| = \text{sqrt}(v1^2 + v2^2 + ... + vn^2)$

```
v = np.array([3, 4])   # A classic 3-4-5 right triangle

# Manual calculation
manual = np.sqrt(v[0]**2 + v[1]**2)
print(manual)                  # 5.0

# NumPy way
print(np.linalg.norm(v))        # 5.0   (L2 norm, default)
print(np.linalg.norm(v, ord=1)) # 7.0   (L1 norm: |3|+|4|=7)
print(np.linalg.norm(v, ord=np.inf))  # 4.0  (Linf norm: max(|3|,|4|))
```

Unit Vectors — Normalizing

A unit vector has length exactly 1. You create one by dividing a vector by its norm. This strips away the magnitude and keeps only direction:

```
v = np.array([3.0, 4.0])

unit_v = v / np.linalg.norm(v)
print(unit_v)                    # [0.6 0.8]
print(np.linalg.norm(unit_v))    # 1.0  — confirmed: length is 1
```

Deep Learning Connection

L2 regularization in neural networks penalizes large weight norms, preventing overfitting. Batch normalization normalizes activations. Gradient clipping limits the gradient norm. Norms are everywhere.

MODULE 2

Matrices

Rectangular grids of numbers that transform the world

2.1 What Is a Matrix?

> **Matrix**
>
> A 2-dimensional grid of numbers arranged in rows and columns. Written as an uppercase bold letter: A. A matrix with m rows and n columns is called an m×n matrix.

Think of a matrix as a stack of row vectors, or as a table of numbers. A 3×4 matrix has 3 rows and 4 columns, giving 12 total numbers.

```
import numpy as np

# Create a 3×4 matrix
A = np.array([
    [1,  2,  3,  4],    # Row 0
    [5,  6,  7,  8],    # Row 1
    [9, 10, 11, 12]     # Row 2
])

print(A.shape)     # (3, 4) — 3 rows, 4 columns
print(A.ndim)      # 2
print(A.size)      # 12 — total elements
print(A.dtype)     # int64
```

2.2 Indexing and Slicing Matrices

```
A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

# Single element: A[row, col]
print(A[0, 0])     # 1   — top-left corner
print(A[1, 2])     # 6   — row 1, col 2
print(A[-1, -1])   # 9   — bottom-right corner

# Entire row
print(A[1, :])     # [4 5 6]  — all of row 1
print(A[1])        # [4 5 6]  — same thing
```

```
# Entire column
print(A[:, 1])    # [2 5 8]  — all of column 1

# Sub-matrix (slicing both dimensions)
print(A[0:2, 0:2])
# [[1 2]
#  [4 5]]
```

2.3 Creating Matrices — The Key Functions

```
import numpy as np

# All zeros — shape is given as a tuple (rows, cols)
Z = np.zeros((3, 4))
print(Z)
# [[0. 0. 0. 0.]
#  [0. 0. 0. 0.]
#  [0. 0. 0. 0.]]

# All ones
O = np.ones((2, 3))

# Identity matrix — square, 1s on the diagonal
I = np.eye(3)
print(I)
# [[1. 0. 0.]
#  [0. 1. 0.]
#  [0. 0. 1.]]

# Random values (uniform between 0 and 1)
R = np.random.rand(3, 3)

# Random values (standard normal: mean=0, std=1)
N = np.random.randn(3, 3)
```

> **Deep Learning Connection**
> Weight matrices in neural networks are initialized with random values (usually from a normal distribution). The np.random.randn pattern is used constantly. Zeros matrices are used for bias initialization.

2.4 Matrix Arithmetic

Element-wise Operations (same shape required)

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print(A + B)    # [[6, 8], [10, 12]]  — add each element
print(A - B)    # [[-4, -4], [-4, -4]]
print(A * B)    # [[ 5, 12], [21, 32]]  — element-wise multiply
print(A / B)    # ratios...
print(A ** 2)   # [[ 1,  4], [ 9, 16]]  — square each element

# Scalar operations broadcast to every element
print(A * 3)    # [[ 3,  6], [ 9, 12]]
print(A + 10)   # [[11, 12], [13, 14]]
```

Visualising a Matrix with Matplotlib (Heatmap)

```
import numpy as np
import matplotlib.pyplot as plt

# Create a 5×5 matrix of random values
M = np.random.rand(5, 5)

plt.figure(figsize=(6, 5))
plt.imshow(M, cmap='Blues')   # 'cmap' sets the color scheme
plt.colorbar()                # Show the color scale
plt.title('Matrix as a Heatmap')

# Annotate each cell with its value
for i in range(5):
    for j in range(5):
        plt.text(j, i, f'{M[i,j]:.2f}',
                 ha='center', va='center', fontsize=9)

plt.show()
```

2.5 Matrix Multiplication — The Core Operation

Matrix Multiplication

Takes two matrices A (m×k) and B (k×n) and produces C (m×n). The number of columns in A must equal the number of rows in B. Each element C[i,j] is the dot product of row i of A with column j of B.

From first principles: if A is 2×3 and B is 3×2, then C = A @ B is 2×2. Think of it as: 'for every combination of a row from A and a column from B, take their dot product.'

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])    # Shape (2, 3)

B = np.array([[7,  8],
              [9,  10],
              [11, 12]])     # Shape (3, 2)

C = A @ B                   # Matrix multiply: shape (2, 2)
# Also: C = np.dot(A, B)    # Same result
# Also: C = np.matmul(A, B) # Same result

print(C)
# [[ 58  64]
#  [139 154]]

# Verify manually for C[0,0]:
# Row 0 of A = [1,2,3], Col 0 of B = [7,9,11]
# Dot product: 1*7 + 2*9 + 3*11 = 7 + 18 + 33 = 58  ✓

print(C.shape)  # (2, 2)
```

CRITICAL Matrix multiplication is NOT commutative! A @ B ≠ B @ A in general. Also, A * B in NumPy is element-wise — NOT matrix multiplication. Always use @ for matrix multiply.

Shape Compatibility Rules — How to Never Get It Wrong

```
# The INNER dimensions must match, the OUTER become the result shape

# (2, 3) @ (3, 5) → (2, 5)  ✓ inner both = 3
# (5, 4) @ (4, 1) → (5, 1)  ✓ inner both = 4
# (2, 3) @ (4, 5) → ERROR   ✗ inner: 3 ≠ 4

A = np.random.randn(2, 3)
B = np.random.randn(3, 5)
C = A @ B
print(f'({A.shape}) @ ({B.shape}) = {C.shape}')
# (2, 3) @ (3, 5) = (2, 5)
```

Deep Learning Connection

A single dense (fully connected) layer in a neural network is: output = input @ W + b, where W is the weight matrix and b is the bias vector. Understanding matrix multiply IS

understanding what a neural network layer does.

2.6 The Transpose

Transpose

Flips a matrix over its diagonal. Rows become columns and columns become rows. If A is (m, n), then A^T is (n, m). In NumPy: A.T

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])

print(A.shape)   # (2, 3)
print(A.T)
# [[1 4]
#  [2 5]
#  [3 6]]
print(A.T.shape) # (3, 2)

# Common use: converting a row vector to a column vector
v = np.array([1, 2, 3])   # shape (3,)
# Note: 1D arrays don't change with .T — use reshape instead
col = v.reshape(-1, 1)    # shape (3, 1) — column vector
```

2.7 Broadcasting — NumPy's Superpower

Broadcasting

NumPy's ability to perform operations on arrays of different shapes by automatically 'stretching' the smaller array to match the larger one, without copying data.

Broadcasting follows rules that are checked from the trailing dimensions. But for beginners, the most common patterns are:

```
import numpy as np

# Pattern 1: Matrix + Scalar (scalar stretches to fill matrix)
A = np.array([[1, 2, 3], [4, 5, 6]])
print(A + 100)
# [[101 102 103]
#  [104 105 106]]

# Pattern 2: Matrix + Row vector (row repeats for each row of matrix)
A = np.array([[1, 2, 3],
```

```
               [4, 5, 6]])
b = np.array([10, 20, 30])  # shape (3,)
print(A + b)                # b is broadcast across both rows
# [[11 22 33]
#  [14 25 36]]

# Pattern 3: Column + Row vector (outer addition)
col = np.array([[1], [2], [3]])   # shape (3, 1)
row = np.array([10, 20, 30])      # shape (3,)
print(col + row)
# [[11 21 31]
#  [12 22 32]
#  [13 23 33]]
```

Deep Learning Connection

Adding biases to neuron outputs uses broadcasting: output = (batch @ W) + b, where output is (batch_size, neurons) and b is (neurons,). NumPy broadcasts b across every sample in the batch automatically.

MODULE 3

Linear Transformations

Seeing what matrices actually do to space

3.1 What Is a Linear Transformation?

> **Linear Transformation**
>
> A function that maps vectors to other vectors using a matrix. When you multiply a matrix A by a vector v, you are transforming (rotating, scaling, shearing, projecting) v into a new vector.

A matrix is not just a table of numbers — it is a recipe for transforming space. Every matrix you multiply by is doing something geometric to your data. Understanding this is the key to understanding why deep learning works.

```
import numpy as np

# A 2x2 matrix transforms 2D vectors
# This matrix SCALES x by 2 and y by 3
A = np.array([[2, 0],
              [0, 3]])

v = np.array([1, 1])     # Original vector
v_transformed = A @ v    # Apply the transformation
print(v_transformed)     # [2 3] — x doubled, y tripled
```

Visualising the Transformation

```
import numpy as np
import matplotlib.pyplot as plt

def plot_transformation(A, title=''):
    # Create a grid of points
    xs = np.linspace(-2, 2, 10)
    ys = np.linspace(-2, 2, 10)
    points = []
    for x in xs:
        for y in ys:
            points.append([x, y])
    points = np.array(points).T   # shape (2, N)

    # Apply transformation
```

```
    transformed = A @ points

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

    ax1.scatter(points[0], points[1], color='navy', s=20)
    ax1.set_title('Before Transformation')
    ax1.set_xlim(-5, 5); ax1.set_ylim(-5, 5)
    ax1.axhline(0, color='gray', lw=0.5)
    ax1.axvline(0, color='gray', lw=0.5)
    ax1.grid(True, alpha=0.3)
    ax1.set_aspect('equal')

    ax2.scatter(transformed[0], transformed[1], color='crimson', s=20)
    ax2.set_title(f'After: {title}')
    ax2.set_xlim(-5, 5); ax2.set_ylim(-5, 5)
    ax2.axhline(0, color='gray', lw=0.5)
    ax2.axvline(0, color='gray', lw=0.5)
    ax2.grid(True, alpha=0.3)
    ax2.set_aspect('equal')

    plt.tight_layout()
    plt.show()

# Scale x by 2, y by 0.5
A = np.array([[2, 0], [0, 0.5]])
plot_transformation(A, 'Scale')

# Rotation by 45 degrees
theta = np.pi / 4
R = np.array([[np.cos(theta), -np.sin(theta)],
              [np.sin(theta),  np.cos(theta)]])
plot_transformation(R, 'Rotation 45°')
```

3.2 Key Special Matrices

The Identity Matrix — Does Nothing

```
I = np.eye(3)
v = np.array([5, -2, 7])
print(I @ v)    # [5. -2.  7.]  — unchanged
# The identity matrix is the number '1' for matrices.
# A @ I = A and I @ A = A for any matrix A
```

Rotation Matrix

```
import numpy as np
import matplotlib.pyplot as plt

def rotation_matrix(angle_degrees):
    theta = np.radians(angle_degrees)  # convert degrees to radians
    return np.array([[np.cos(theta), -np.sin(theta)],
                     [np.sin(theta),  np.cos(theta)]])

v = np.array([1, 0])   # Vector pointing right

fig, ax = plt.subplots(figsize=(6, 6))

colors = ['navy', 'blue', 'dodgerblue', 'gold', 'orange']
for i, angle in enumerate([0, 45, 90, 135, 180]):
    R = rotation_matrix(angle)
    rotated = R @ v
    ax.quiver(0, 0, rotated[0], rotated[1],
              angles='xy', scale_units='xy', scale=1,
              color=colors[i], label=f'{angle}°')

ax.set_xlim(-1.5, 1.5)
ax.set_ylim(-1.5, 1.5)
ax.set_aspect('equal')
ax.legend()
ax.grid(True, alpha=0.3)
ax.set_title('Rotating a vector by different angles')
plt.show()
```

3.3 The Determinant — Does This Transformation Flip Space?

> **Determinant**
>
> A single number computed from a square matrix. It tells you how much a transformation scales areas (in 2D) or volumes (in 3D). If det = 0, the matrix is singular and cannot be inverted.

```
import numpy as np

A = np.array([[2, 0],
              [0, 3]])
print(np.linalg.det(A))   # 6.0
# This transformation scales area by 6x

B = np.array([[1, 2],
              [2, 4]])     # Row 1 = 2 * Row 0 (linearly dependent!)
print(np.linalg.det(B))   # ~0.0 (singular matrix)
```

```
# Rotation matrices always have determinant = 1 (no scaling)
theta = np.pi / 3
R = np.array([[np.cos(theta), -np.sin(theta)],
              [np.sin(theta),  np.cos(theta)]])
print(np.linalg.det(R))   # 1.0
```

3.4 The Inverse — Undoing a Transformation

Matrix Inverse

For a square matrix A, its inverse A^{-1} is the matrix that undoes A's transformation. $A @ A^{-1} = I$. Only exists if $\det(A) \neq 0$.

```
import numpy as np

A = np.array([[2., 1.],
              [5., 3.]])

A_inv = np.linalg.inv(A)
print(A_inv)
# [[ 3. -1.]
#  [-5.  2.]]

# Verify: A @ A_inv should equal identity
print(np.round(A @ A_inv, decimals=10))
# [[1. 0.]
#  [0. 1.]]   ✓

# Practical use: solving Ax = b
b = np.array([3., 7.])
x = A_inv @ b       # x = A_inv @ b
print(x)            # The solution to the linear system

# Better way to solve linear systems (more numerically stable):
x = np.linalg.solve(A, b)
print(x)
```

Deep Learning Connection

In practice, deep learning rarely computes explicit matrix inverses (they are expensive and numerically unstable). Instead, gradients are used to iteratively adjust weights. But understanding the inverse concept helps you understand why some network architectures are poorly conditioned.

MODULE 4

Eigenvalues, SVD & PCA

The deep structure of matrices — and why it matters for AI

4.1 Eigenvectors and Eigenvalues — The Special Directions

Eigenvector & Eigenvalue

For a square matrix A, an eigenvector v is a special vector that, when transformed by A, only gets scaled (not rotated). The scale factor is called the eigenvalue λ. In math: A @ v = λ * v

Most vectors change direction when you multiply by a matrix. Eigenvectors are the special exceptions — they land on the same line they started on, just stretched or squashed by λ.

```
import numpy as np

A = np.array([[3, 1],
              [0, 2]])

# np.linalg.eig returns (eigenvalues, eigenvectors)
# Each COLUMN of the matrix is one eigenvector
eigenvalues, eigenvectors = np.linalg.eig(A)

print('Eigenvalues:', eigenvalues)
# Eigenvalues: [3. 2.]

print('Eigenvectors (columns):',)
print(eigenvectors)
# [[1.         0.70710678]
#  [0.         0.70710678]]

# Verify: A @ v = lambda * v for the first eigenpair
v0 = eigenvectors[:, 0]          # First eigenvector
lam0 = eigenvalues[0]            # First eigenvalue
print(np.allclose(A @ v0, lam0 * v0))  # True ✓
```

Visualising Eigenvectors

```
import numpy as np
import matplotlib.pyplot as plt
```

```
A = np.array([[3., 1.],
              [0., 2.]])

vals, vecs = np.linalg.eig(A)

fig, ax = plt.subplots(figsize=(7, 7))

# Show effect on random vectors
np.random.seed(42)
for _ in range(12):
    v = np.random.randn(2)
    v = v / np.linalg.norm(v)     # unit vector
    Av = A @ v
    ax.annotate('', xy=Av, xytext=[0,0],
                arrowprops=dict(arrowstyle='->', color='lightgray'))
    ax.annotate('', xy=v, xytext=[0,0],
                arrowprops=dict(arrowstyle='->', color='navy', lw=1.5))

# Show eigenvectors
for i, (val, vec) in enumerate(zip(vals, vecs.T)):
    vec = vec / np.linalg.norm(vec)
    ax.annotate('', xy=val*vec, xytext=[0,0],
                arrowprops=dict(arrowstyle='->', color='gold', lw=3))
    ax.text(val*vec[0]*1.1, val*vec[1]*1.1,
            f'λ={val:.1f}', fontsize=12, color='darkgoldenrod')

ax.set_xlim(-4, 4); ax.set_ylim(-4, 4)
ax.axhline(0, color='k', lw=0.5); ax.axvline(0, color='k', lw=0.5)
ax.grid(True, alpha=0.3); ax.set_aspect('equal')
ax.set_title('Eigenvectors (gold) vs random vectors (navy → gray)')
plt.show()
```

Deep Learning Connection

The convergence of gradient descent depends on the eigenvalues of the Hessian matrix. Large eigenvalues cause instability; small ones slow learning. The optimal learning rate is related to the largest eigenvalue. Adam optimizer accounts for this implicitly.

4.2 SVD — Singular Value Decomposition

SVD

Any matrix A (m×n) can be decomposed into three matrices: A = U @ Σ @ V^T, where U and V are orthogonal matrices (rotations) and Σ is diagonal (scales). The diagonal values of Σ are the singular values.

SVD is the most important decomposition in data science and machine learning. It reveals the 'essence' of a matrix — what it really does, stripped to its core.

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]], dtype=float)

U, s, Vt = np.linalg.svd(A)

print('U shape:', U.shape)    # (3, 3) — left singular vectors
print('s shape:', s.shape)    # (3,)   — singular values
print('Vt shape:', Vt.shape)  # (3, 3) — right singular vectors (transposed)

print('Singular values:', np.round(s, 4))
# [1.6848e+01  1.0684e+00  3.5762e-16]
# The near-zero last value tells us this matrix is rank-deficient!

# Reconstruct A from SVD
Sigma = np.diag(s)
A_reconstructed = U @ Sigma @ Vt
print(np.allclose(A, A_reconstructed))  # True ✓
```

Low-Rank Approximation with SVD — Image Compression

```
import numpy as np
import matplotlib.pyplot as plt

# Create a 'fake image' matrix for demonstration
np.random.seed(0)
A = np.outer(np.sin(np.linspace(0, np.pi, 100)),
             np.cos(np.linspace(0, np.pi, 100)))
A += 0.1 * np.random.randn(100, 100)

U, s, Vt = np.linalg.svd(A)

fig, axes = plt.subplots(1, 4, figsize=(16, 4))

axes[0].imshow(A, cmap='viridis'); axes[0].set_title('Original')

for ax, k in zip(axes[1:], [1, 5, 20]):
    # Keep only the top-k singular values
    A_approx = U[:, :k] @ np.diag(s[:k]) @ Vt[:k, :]
    ax.imshow(A_approx, cmap='viridis')
    ax.set_title(f'Rank-{k} approximation')
    ax.set_xlabel(f'{k}/{len(s)} singular values')
```

```
plt.tight_layout()
plt.show()
```

Deep Learning Connection

SVD is the mathematical foundation of recommendation systems (Netflix, Spotify). It is also used in weight compression for large language models, and is related to Principal Component Analysis — the next topic.

4.3 PCA — Principal Component Analysis

PCA

A technique that finds the directions of maximum variance in a dataset and projects the data onto those directions. It reduces high-dimensional data to lower dimensions while losing as little information as possible.

PCA from first principles uses SVD. Here is the complete workflow:

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Create 2D data with a clear 'main direction'
np.random.seed(42)
mean = [2, 3]
cov = [[3, 2], [2, 2]]          # Covariance defines the shape/tilt
X = np.random.multivariate_normal(mean, cov, 300)  # 300 points, 2D

# Step 2: Center the data (subtract mean)
X_centered = X - X.mean(axis=0)

# Step 3: Compute SVD on the centered data
U, s, Vt = np.linalg.svd(X_centered, full_matrices=False)

# Step 4: Principal components are the rows of Vt
PC1 = Vt[0]   # Direction of most variance
PC2 = Vt[1]   # Direction of second most variance (perpendicular to PC1)

# Step 5: Project data onto principal components
X_pca = X_centered @ Vt.T  # shape (300, 2)

# Visualise
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
```

```
ax1.scatter(X[:,0], X[:,1], alpha=0.5, color='navy', s=15)
# Draw principal component directions
scale = s[0] / np.sqrt(len(X)) * 2
ax1.quiver(*X.mean(axis=0), *PC1*scale, color='gold', scale=1,
           scale_units='xy', angles='xy', width=0.01)
ax1.set_title('Original 2D Data with PC1 (gold)')
ax1.set_aspect('equal')
ax1.grid(True, alpha=0.3)

ax2.scatter(X_pca[:,0], X_pca[:,1], alpha=0.5, color='crimson', s=15)
ax2.set_xlabel('Principal Component 1')
ax2.set_ylabel('Principal Component 2')
ax2.set_title('Data in PCA Space')
ax2.grid(True, alpha=0.3)

# Explained variance
explained_var = s**2 / np.sum(s**2)
print(f'PC1 explains: {explained_var[0]*100:.1f}% of variance')
print(f'PC2 explains: {explained_var[1]*100:.1f}% of variance')

plt.tight_layout()
plt.show()
```

Deep Learning Connection

PCA is used as a preprocessing step before neural networks to reduce input dimensionality. It also inspired modern attention mechanisms: transformers learn 'which dimensions of the embedding space to attend to', which is conceptually related to finding principal components dynamically.

MODULE 5

Gradients & Optimization

Where linear algebra meets calculus for deep learning

5.1 The Gradient — A Generalised Derivative

> **Gradient**
>
> For a function that takes a vector input and outputs a scalar, the gradient is a vector of partial derivatives — one per input dimension. It points in the direction of steepest ascent.

In single-variable calculus, the derivative tells you 'which direction to move on a 1D number line to increase the function'. The gradient generalises this to multiple dimensions. For a neural network with millions of parameters, the gradient is a vector with millions of components.

```
import numpy as np
import matplotlib.pyplot as plt

# Example: f(x, y) = x² + y²  (a bowl shape — paraboloid)
# Gradient: ∇f = [df/dx, df/dy] = [2x, 2y]

def f(x, y):
    return x**2 + y**2

def gradient_f(x, y):
    return np.array([2*x, 2*y])    # Analytical gradient

# Numerical gradient approximation (how autograd works conceptually)
def numerical_gradient(f, x, y, h=1e-5):
    df_dx = (f(x+h, y) - f(x-h, y)) / (2*h)   # Central difference
    df_dy = (f(x, y+h) - f(x, y-h)) / (2*h)
    return np.array([df_dx, df_dy])

# Test at point (1, 2)
print('Analytical gradient at (1,2):', gradient_f(1, 2))
# [2. 4.]
print('Numerical gradient at (1,2):', numerical_gradient(f, 1, 2))
# [2. 4.]  — same! ✓
```

Visualising the Gradient Field

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Create a grid of (x, y) values
x = np.linspace(-3, 3, 20)
y = np.linspace(-3, 3, 20)
X, Y = np.meshgrid(x, y)    # meshgrid creates 2D coordinate arrays

# f(x, y) = x^2 + 2*y^2  (an elliptic paraboloid)
Z = X**2 + 2*Y**2

# Gradients: df/dx = 2x, df/dy = 4y
U = 2 * X     # x-component of gradient
V = 4 * Y     # y-component of gradient

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Contour plot (showing the 'height' of f)
axes[0].contourf(X, Y, Z, levels=20, cmap='Blues')
axes[0].contour(X, Y, Z, levels=20, colors='navy', alpha=0.3)
axes[0].set_title('f(x,y) = x² + 2y² — Contour Map')
axes[0].set_xlabel('x'); axes[0].set_ylabel('y')

# Gradient field (arrows showing direction of steepest ascent)
axes[1].contourf(X, Y, Z, levels=20, cmap='Blues', alpha=0.4)
axes[1].quiver(X, Y, U, V, color='crimson', alpha=0.7)
axes[1].set_title('Gradient Field ∇f — Arrows Point Uphill')
axes[1].set_xlabel('x'); axes[1].set_ylabel('y')

plt.tight_layout()
plt.show()
```

5.2 Gradient Descent — How Neural Networks Learn

> **Gradient Descent**
> An iterative algorithm that minimises a function by repeatedly moving in the direction opposite to the gradient. Like rolling a ball downhill to find the valley.

The update rule is: new_params = old_params - learning_rate × gradient. The learning rate controls how large each step is.

```
import numpy as np
import matplotlib.pyplot as plt

# Minimise f(x) = (x - 3)^2  (minimum at x = 3)
def f(x):        return (x - 3)**2
def df(x):       return 2 * (x - 3)    # Derivative: df/dx
```

```
# Gradient descent loop
x = -2.0          # Starting point (far from minimum)
lr = 0.1          # Learning rate
history = [x]     # Record the path

for step in range(30):
    grad = df(x)          # Compute gradient
    x = x - lr * grad     # Move OPPOSITE to gradient (downhill)
    history.append(x)
    if step < 5:          # Print first few steps
        print(f'Step {step+1}: x={x:.4f}, f(x)={f(x):.6f}')

# Visualise the descent
x_range = np.linspace(-3, 7, 200)
plt.figure(figsize=(10, 5))
plt.plot(x_range, f(x_range), 'navy', lw=2, label='f(x) = (x-3)²')
plt.scatter(history, [f(xi) for xi in history],
            c=range(len(history)), cmap='autumn_r',
            s=50, zorder=5, label='Descent path')
plt.axvline(3, color='green', ls='--', alpha=0.5, label='Minimum at x=3')
plt.xlabel('x'); plt.ylabel('f(x)')
plt.title('Gradient Descent Converging to Minimum')
plt.legend(); plt.grid(True, alpha=0.3)
plt.show()
```

2D Gradient Descent — The Neural Network Case

```
import numpy as np
import matplotlib.pyplot as plt

# f(w1, w2) = w1^2 + 5*w2^2  (like a loss surface with 2 weights)
def f(w):  return w[0]**2 + 5*w[1]**2
def df(w): return np.array([2*w[0], 10*w[1]])

w = np.array([-2.5, 1.5])  # Starting weights
lr = 0.08
path = [w.copy()]

for _ in range(50):
    w = w - lr * df(w)
    path.append(w.copy())

path = np.array(path)

# Plot loss surface and descent path
w1 = np.linspace(-3, 3, 100)
```

```
w2 = np.linspace(-2, 2, 100)
W1, W2 = np.meshgrid(w1, w2)
Z = W1**2 + 5*W2**2

plt.figure(figsize=(8, 6))
plt.contourf(W1, W2, Z, levels=30, cmap='Blues', alpha=0.7)
plt.contour(W1, W2, Z, levels=30, colors='navy', alpha=0.3)
plt.plot(path[:,0], path[:,1], 'o-', color='gold',
         markersize=4, lw=2, label='Gradient descent path')
plt.scatter([0], [0], color='red', s=100, zorder=5,
            marker='*', label='Minimum (0,0)')
plt.xlabel('w1 (weight 1)'); plt.ylabel('w2 (weight 2)')
plt.title('Gradient Descent on a 2D Loss Surface')
plt.colorbar(label='Loss')
plt.legend()
plt.show()
```

5.3 The Jacobian — Gradients of Vector Functions

Jacobian

When your function maps a vector to a vector (not a scalar), the Jacobian is a matrix of all partial derivatives. It is the matrix version of the derivative. Shape: (output_dim, input_dim).

```
import numpy as np

# Function f: R^2 -> R^2
# f([x, y]) = [x^2 + y, x*y - y^2]

# Jacobian (computed analytically):
# df1/dx = 2x,   df1/dy = 1
# df2/dx = y,    df2/dy = x - 2y

def jacobian(x, y):
    return np.array([[2*x, 1],
                     [y,   x - 2*y]])

# At point (1, 2):
J = jacobian(1, 2)
print(J)
# [[2 1]
#  [2 -3]]

# The Jacobian tells us: 'How does each output change with each input?'
# This is exactly what backpropagation computes through every layer
```

Deep Learning Connection

Backpropagation is just the chain rule applied to Jacobians. When gradients flow backwards through a neural network, each layer computes a Jacobian-vector product. Modern deep learning frameworks (PyTorch, JAX) compute this efficiently using automatic differentiation.

MODULE 6

Building a Neural Network Layer from Scratch

Every concept from Modules 1–5, unified into one working example

In this final module we build a single neural network layer from absolute scratch using only NumPy — no PyTorch, no TensorFlow. Every line of math we use comes from this course. This is the payoff.

6.1 What a Neural Network Layer Does

A single dense (fully connected) layer does exactly three things:

1. **Linear transformation:** z = X @ W + b (matrix multiply + bias broadcast)
2. **Activation function:** a = activation(z) (element-wise nonlinearity)
3. **Backward pass:** compute gradients with respect to W and b

Let us build this step by step:

6.2 Forward Pass

```
import numpy as np
import matplotlib.pyplot as plt

# ── Setup ──────────────────────────────────────────────────────────
np.random.seed(42)

# Simulated dataset: 100 samples, 4 input features each
batch_size = 100
input_dim  = 4
output_dim = 3   # This layer has 3 neurons

X = np.random.randn(batch_size, input_dim)   # shape: (100, 4)

# ── Layer Parameters ───────────────────────────────────────────────
# He initialization (good for ReLU): scale by sqrt(2/input_dim)
W = np.random.randn(input_dim, output_dim) * np.sqrt(2.0 / input_dim)
b = np.zeros((1, output_dim))   # shape: (1, 3) — broadcasts over batch

print('X shape:', X.shape)  # (100, 4)
print('W shape:', W.shape)  # (4, 3)
print('b shape:', b.shape)  # (1, 3)

# ── Linear Step ────────────────────────────────────────────────────
```

```
Z = X @ W + b               # shape: (100, 3)
# X @ W: (100,4) @ (4,3) = (100,3)  — each sample gets 3 outputs
# + b:   broadcasts (1,3) across 100 rows
print('Z shape:', Z.shape)  # (100, 3)

# ── Activation: ReLU ──────────────────────────────────────────
# ReLU: f(x) = max(0, x)  — kills negative values
A = np.maximum(0, Z)        # element-wise max with 0
print('A shape:', A.shape)  # (100, 3)
```

6.3 Activation Functions — Visualised

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-4, 4, 300)

# Define common activations
relu    = np.maximum(0, x)
sigmoid = 1 / (1 + np.exp(-x))
tanh    = np.tanh(x)
leaky   = np.where(x > 0, x, 0.1 * x)

fig, axes = plt.subplots(2, 2, figsize=(12, 8))

configs = [
    (relu,    'ReLU: max(0, x)',          'navy',   'Most common in hidden
layers'),
    (sigmoid, 'Sigmoid: 1/(1+e^-x)',      'crimson','Output layer for binary
classification'),
    (tanh,    'Tanh: (e^x - e^-x)/(e^x + e^-x)', 'green', 'RNNs — zero-centered'),
    (leaky,   'Leaky ReLU: x if x>0 else 0.1x',  'purple','Avoids dying ReLU
problem'),
]

for ax, (y, name, color, note) in zip(axes.flat, configs):
    ax.plot(x, y, color=color, lw=2.5)
    ax.axhline(0, color='gray', lw=0.5)
    ax.axvline(0, color='gray', lw=0.5)
    ax.set_title(name, fontsize=12, fontweight='bold')
    ax.set_xlabel(note, fontsize=9, color='gray')
    ax.grid(True, alpha=0.3)
    ax.set_ylim(-1.5, 1.5)

plt.tight_layout()
plt.show()
```

6.4 Backward Pass — Computing Gradients

Now we compute how much each weight contributed to the error, so we know how to adjust them. This is backpropagation — and it is pure linear algebra.

```
# Continuing from the forward pass above...

# Simulated: suppose we have targets and compute a simple MSE loss
y_true = np.random.randn(batch_size, output_dim)  # shape (100, 3)

# Loss: Mean Squared Error = mean((A - y_true)^2)
loss = np.mean((A - y_true)**2)
print(f'Loss: {loss:.4f}')

# ── Backward Pass ──────────────────────────────────────────────────
# Step 1: Gradient of loss w.r.t. A
dL_dA = 2 * (A - y_true) / (batch_size * output_dim)  # shape (100, 3)

# Step 2: Gradient through ReLU
# ReLU gradient: 1 where Z > 0, else 0
dL_dZ = dL_dA * (Z > 0)   # element-wise mask  shape (100, 3)

# Step 3: Gradient w.r.t. W
# Z = X @ W, so dZ/dW involves X
# dL/dW = X^T @ dL_dZ
dL_dW = X.T @ dL_dZ   # (4, 100) @ (100, 3) = (4, 3) — same shape as W!

# Step 4: Gradient w.r.t. b
# dL/db = sum over batch dimension
dL_db = np.sum(dL_dZ, axis=0, keepdims=True)  # (1, 3) — same shape as b!

print('dL/dW shape:', dL_dW.shape)  # (4, 3) ✓
print('dL/db shape:', dL_db.shape)  # (1, 3) ✓

# ── Weight Update (Gradient Descent) ──────────────────────────────
lr = 0.01
W = W - lr * dL_dW
b = b - lr * dL_db
print('Weights updated!')
```

6.5 Full Training Loop

```
import numpy as np
import matplotlib.pyplot as plt
```

```
np.random.seed(0)

# ── Dataset: Learn the function y = sin(x) ──────────────────────
X = np.linspace(-np.pi, np.pi, 200).reshape(-1, 1)  # (200, 1)
y = np.sin(X)                                        # (200, 1)

# ── Network: 1 input → 32 hidden → 1 output ────────────────────
W1 = np.random.randn(1, 32) * 0.5
b1 = np.zeros((1, 32))
W2 = np.random.randn(32, 1) * 0.5
b2 = np.zeros((1, 1))

lr = 0.005
losses = []

for epoch in range(3000):
    # Forward
    Z1 = X @ W1 + b1          # (200, 32)
    A1 = np.tanh(Z1)          # tanh activation
    Z2 = A1 @ W2 + b2         # (200, 1)
    A2 = Z2                   # Linear output

    loss = np.mean((A2 - y)**2)
    losses.append(loss)

    # Backward
    dL_dA2 = 2 * (A2 - y) / len(X)
    dL_dW2 = A1.T @ dL_dA2
    dL_db2 = np.sum(dL_dA2, axis=0, keepdims=True)

    dL_dA1 = dL_dA2 @ W2.T
    dL_dZ1 = dL_dA1 * (1 - np.tanh(Z1)**2)  # tanh gradient
    dL_dW1 = X.T @ dL_dZ1
    dL_db1 = np.sum(dL_dZ1, axis=0, keepdims=True)

    # Update
    W1 -= lr * dL_dW1; b1 -= lr * dL_db1
    W2 -= lr * dL_dW2; b2 -= lr * dL_db2

# ── Visualise Results ──────────────────────────────────────────
Z1_final = X @ W1 + b1
A1_final = np.tanh(Z1_final)
y_pred   = A1_final @ W2 + b2

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

ax1.semilogy(losses, color='navy', lw=1.5)
ax1.set_xlabel('Epoch'); ax1.set_ylabel('MSE Loss (log scale)')
```

```
ax1.set_title('Training Loss')
ax1.grid(True, alpha=0.3)

ax2.plot(X, y, 'navy', lw=2, label='True: sin(x)')
ax2.plot(X, y_pred, 'gold', lw=2, ls='--', label='Network prediction')
ax2.set_xlabel('x'); ax2.set_ylabel('y')
ax2.set_title('Network Learning sin(x)')
ax2.legend(); ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
print(f'Final loss: {losses[-1]:.6f}')
```

CONGRATULATIONS	You just implemented a complete neural network from scratch using nothing but NumPy and the linear algebra concepts from this course. Every operation — matrix multiply, gradient, dot product, transpose — you have learned from first principles.

6.6 NumPy & Matplotlib Quick Reference

Essential NumPy commands:

```
# Array creation
np.array([...])            # from list
np.zeros((m, n))           # zeros matrix
np.ones((m, n))            # ones matrix
np.eye(n)                  # identity matrix
np.random.randn(m, n)      # normal random
np.linspace(start,stop,n)  # evenly spaced
np.arange(start,stop,step)# range of values

# Shape & info
a.shape    # dimensions tuple
a.ndim     # number of dimensions
a.dtype    # data type
a.size     # total number of elements

# Reshaping
a.reshape(m, n)      # change shape (total elements must match)
a.reshape(-1, 1)     # column vector (-1 = 'figure it out')
a.flatten()          # collapse to 1D
a.T                  # transpose

# Math
a @ b                # matrix multiply
```

```
np.dot(a, b)        # dot product / matrix multiply
a * b               # element-wise multiply
np.sum(a, axis=0)   # sum across rows (result: one row)
np.sum(a, axis=1)   # sum across cols (result: one col)
np.mean(a)          # mean of all elements
np.std(a)           # standard deviation
np.max(a)           # maximum
np.abs(a)           # absolute value
np.sqrt(a)          # square root
np.exp(a)           # e^a element-wise
np.log(a)           # natural log element-wise

# Linear algebra
np.linalg.norm(v)        # vector length
np.linalg.det(A)         # determinant
np.linalg.inv(A)         # inverse
np.linalg.solve(A, b)    # solve Ax = b
np.linalg.eig(A)         # eigenvalues and vectors
np.linalg.svd(A)         # SVD decomposition
```

Essential Matplotlib commands:

```
import matplotlib.pyplot as plt

# Figure setup
plt.figure(figsize=(w, h))
fig, ax = plt.subplots(rows, cols, figsize=(w, h))

# Line/scatter plots
plt.plot(x, y, color='navy', lw=2, ls='--', label='name')
plt.scatter(x, y, color='red', s=50, alpha=0.7)

# Matrix/image display
plt.imshow(M, cmap='viridis')       # show 2D array as image
plt.colorbar()                       # show color scale

# Vector plots
plt.quiver(x0, y0, dx, dy, angles='xy', scale_units='xy', scale=1)

# Contour plots
plt.contourf(X, Y, Z, levels=20, cmap='Blues')  # filled contours
plt.contour(X, Y, Z, levels=20)                 # contour lines

# Labels & formatting
plt.title('...')
plt.xlabel('...')
plt.ylabel('...')
plt.legend()
```

```
plt.grid(True, alpha=0.3)
plt.tight_layout()    # prevent label clipping
plt.show()            # display the plot
```